Generating the full TinyWorlds dataset is proving to be extremely frustrating, I cant get a contemporary model to match the same characteristics as the original GPT-4 dataset.

I wondering instead about the feasibility of partitioning the original dataset in such a way that I effectively get partly novel, partly in-distribution stories that will let me do the same things as TW was supposed to. This would entail some automatic partioning of the original dataset and then

Show more

Worked for 20m 5s

Recommendation: TinyWorlds-P — Partitioned TinyStories

This is a much better route than trying to reproduce the TinyStories generator.

The original TinyStories generation process already gives you useful latent structure: stories were generated from randomly selected nouns, verbs, adjectives, and optional narrative features such as dialogue, plot twists, morals, or unhappy endings. The released corpus also includes an archive containing the generation metadata and original prompts. arXiv +1

That means you can construct a controlled continual-learning benchmark from **unaltered, genuine TinyStories text**:

- Every story remains fully in-distribution stylistically.
- Every individual noun, verb, adjective, and narrative feature remains familiar to the base model.
- Only selected **combinations** of those factors are absent from base training.
- The continual stream then introduces those missing combinations sequentially.

This is not merely an ad hoc trick. The TinyStories paper itself evaluated an out-of-distribution split made by withholding combinations of instruction types while retaining their individual components. arXiv

I would treat this as a deliberately narrower benchmark than the previous TinyWorlds proposals. It tests the core VAMP claims—retention, task-free address selection, transfer, compositional reuse, memory growth, and addressing cost—without claiming to test exact symbolic world knowledge.

---

1. Partition on generation recipes, not story embeddings

Do **not** cluster the story texts with an embedding model. That would tend to produce broad topical or stylistic domains, making the worlds either trivially routable or genuinely out-of-distribution.

Instead, use the original generation metadata as exogenous variables:

$$r_s = (n_s, v_s, a_s, f_s, q_s),$$

where:

- n_s: forced noun
- v_s: forced verb
- a_s: forced adjective
- f_s: narrative-feature mask
- q_s: generation source or version, where applicable

The text shown to the model is still only the story. Metadata is used solely to construct and audit the split.

Preserve your exact current corpus

Use whichever exact text file your existing TinyStories model trains on as the canonical corpus—original TinyStories or the official GPT-4-only variant.

The metadata archive is a superset, so:

1. Normalize each story conservatively: Unicode normalization, line endings, repeated whitespace, and quotation-mark variants.
2. Compute a SHA-256 hash of the normalized text.

3. Hash-join your current corpus to the metadata archive.
4. Partition only matched stories.
5. Group identical hashes so an exact duplicate can never cross base/world or train/test boundaries.

The official release includes the original train data, separate GPT-4-generated files, and an approximately 1.61 GB metadata archive containing prompts and metadata. 🤗 Hugging Face +1

Do not spend time on semantic near-deduplication initially. Exact normalized-text grouping is enough for the first benchmark.

---

2. Turn noun–verb combinations into “worlds”

Partition the forced-noun vocabulary into eight buckets:

$$N_0, \dots, N_7$$

and the forced-verb vocabulary into eight buckets:

$$V_0, \dots, V_7.$$

Use **frequency-balanced random assignment**, not semantic clustering:

1. Count the number of tokens or stories associated with every noun and verb.
2. Sort words from most to least frequent.
3. Greedily assign each word to the currently smallest bucket.
4. Use a fixed random seed to break ties.

A world is then one cell in an 8×8 recipe grid:

$$W_{ij} = \{s : n_s \in N_i,\ v_s \in V_j\}.$$

The base model sees all nouns and verbs, including those in each held-out cell, but never sees those particular bucket combinations.

For example, purely illustratively:

- N_0: cat, prince, garden, boat, …
- N_1: robot, girl, forest, dragon, …
- V_0: find, carry, hide, remember, …
- V_1: help, open, chase, repair, …

The base corpus may contain `cat + help`, `robot + find`, `prince + repair`, and `girl + carry`, while the entire family `N0 + V0` is absent. Later, the $N_0 \times V_0$ stream consists entirely of genuine original TinyStories—not newly generated approximations.

Adjectives, narrative features, lengths, and model source should be **stratified within every cell**, rather than used as additional world labels. That prevents one world from accidentally becoming "the dialogue world" or "the sad-ending world."

---

3. Use a five-world stream with an explicit missing-corner test

Choose five held-out cells:

Episode	Cell	Purpose
A	$N_0 \times V_0$	Initial novel conjunction
B	$N_1 \times V_0$	Shares the verb-side factor with A
C	$N_1 \times V_1$	Shares the noun-side factor with B
D	$N_0 \times V_1$	Missing corner of the 2×2 square
E	$N_2 \times V_2$	Relatively unrelated control
return A	$N_0 \times V_0$	Recall and recovery test

The stream is therefore:

$$A \to B \to C \to D \to E \to A.$$

This gives you the main TinyWorlds properties without inventing a complicated task ontology:

- $A \to B \to C$: related but distinguishable distribution shifts.
- D: compositional transfer and graph reuse.
- E: unrelated interference or negative transfer.
- Return to A: stored recall versus forgetting.

The key VAMP composition test

Train A, B, and C as base-rooted LoRA edges:

$$\theta_A = \theta_0 + \Delta_A, \qquad \theta_B = \theta_0 + \Delta_B, \qquad \theta_C = \theta_0 + \Delta_C.$$

If the learned deltas approximately factor noun-side and verb-side regularities, the missing corner should be approximated by:

$$\hat{\theta}_D = \theta_0 + \Delta_A - \Delta_B + \Delta_C.$$

Under an additive-factor interpretation,

$$(N_0 + V_0) - (N_1 + V_0) + (N_1 + V_1) = N_0 + V_1.$$

This gives an unusually clean test of whether the LoRA atoms contain reusable structure rather than merely memorizing unrelated domains.

At episode D:

1. Evaluate the frozen base on D.
2. Evaluate each single prior edge.
3. Evaluate the composed address
$$\Delta_A - \Delta_B + \Delta_C.$$
4. Measure zero-shot D loss before any D training.
5. Train a small residual edge:
$$\theta_D = \theta_0 + \Delta_A - \Delta_B + \Delta_C + \rho_D.$$

The composition does not have to be permanently merged. Your VAMP evaluator can apply the three frozen LoRA deltas with coefficients $+1, -1, +1$. If desired, the factors can be concatenated exactly into a temporary higher-rank adapter.

This creates a real DAG:

- root θ_0
- base-rooted atoms A, B, C, E
- a composite D macro-node
- a residual edge ρ_D from the composite node

That is much more informative than five arbitrary topic adapters, while still remaining straightforward.

4. Dataset sizes

With eight noun buckets and eight verb buckets, there are 64 cells.

Holding out five cells removes approximately:

$$\frac{5}{64} = 7.8125\%$$

of the base-training corpus.

Using your existing estimate of approximately 465–474 million tokens:

Partition	Approximate tokens
Reduced base corpus, $59/64$	428.7–437.0M
One complete world cell, $1/64$	7.27–7.41M
World training portion, 80%	5.81–5.93M
World validation portion, 10%	0.73–0.74M
World test portion, 10%	0.73–0.74M

Actual cells will not be perfectly equal, so target no more than roughly ±10% variation in token count.

For each world, split by story hash and stratify on:

- generation source
- narrative-feature signature
- adjective bucket
- story-length bin

The model never receives the bucket IDs.

Matched controls

For W_{ij}, construct an equal-sized seen control from:

- stories in noun row N_i but other verb columns, and
- stories in verb column V_j but other noun rows.

This gives a control containing the same component marginals without the withheld conjunction. The difference

$$L_{\text{held-out conjunction}} - L_{\text{matched seen control}}$$

is a much cleaner measure of novelty than comparison with a random base sample.

---

5. Minimal training matrix

Start with rank 4 only. Run rank 2 after the benchmark behaves sensibly.

Baseline 1: one mutable sequential LoRA

Use one rank-4 adapter and update it continuously:

$$A \to B \to C \to D \to E.$$

After every episode, evaluate it on every world encountered so far.

This is the single-mutable-parameter-state baseline and should reveal whether the stream actually creates interference.

Baseline 2: independent frozen LoRA bank

Train a fresh base-rooted adapter for each world and freeze it.

Use the known world identity to select the correct adapter at evaluation. This is the oracle parameter-isolation baseline:

- It should have essentially no parameter forgetting.
- It does not solve task-free addressing.
- It does not exploit D's missing-corner structure.

Method 3: VAMP LoRA graph

Use:

- base-rooted frozen edges for A, B, C
- the composed $A - B + C$ node plus residual ρ_D
- a base-rooted edge for unrelated E
- task-free energy-based address selection

Do not add replay, consolidation, a learned router, or semantic indexing in the first run.

One inexpensive offline reference run can train a single adapter on shuffled $A \cup B \cup C \cup D \cup E$, giving the sequential-versus-joint gap.

This directly instantiates the minimal addressed-parameter-memory experiment proposed in your paper: compare one mutable state, retained low-rank delta atoms, addressed inference, transfer, memory growth, and addressing cost. Addressed_Parameter_Memories

---

6. Fix the previous prefix-label problem by separating storage from routing

Do not ask a classifier to infer the metadata world label from an arbitrary short prefix. That recreates the exact failure from the earlier TinyWorlds attempt.

Evaluate two separate questions.

Oracle-address evaluation

Use the known assigned world address. This answers:

> Is the competence still represented in the parameter graph?

It cleanly measures adapter storage and parameter interference.

Task-free prefix-energy routing

For a test story, use an observed prefix P, evaluate each candidate address, and choose:

$$a^*(P) = \arg\min_a \left[-\sum_{t \in P} \log p_{\theta(a)}(x_t \mid x_{<t}) \right].$$

Then score only the subsequent continuation C:

$$L_{\text{routed}}(C) = -\sum_{t \in C} \log p_{\theta(a^*(P))}(x_t \mid x_{<t}).$$

A straightforward evaluation window is:

- first 96 tokens: addressing prefix

- next 96 tokens: scored continuation
- stories too short for at least 32+32 tokens remain in oracle LM evaluation but are omitted from the routing test

This is legitimate task-free inference. If the prefix contains insufficient information, that becomes measurable addressing ambiguity rather than an erroneous label assignment.

Report:

- oracle-world NLL
- task-free routed NLL
- routing regret against the best candidate address
- address-selection frequency
- number of candidate forward passes
- fraction of prefixes containing zero, one, or both forced recipe words

The final visibility statistic is diagnostic only; do not filter the headline task-free result to prefixes that expose the answer.

With five worlds plus the base and the D composition, exhaustive routing involves at most about seven tiny-model prefix evaluations. That is intentionally acceptable for the first VAMP experiment. A learned proposal router can wait.

---

7. Metrics that answer the actual CL questions

The core result should be a phase-by-phase matrix of world losses.

Property	Measurement
Acquisition	NLL improvement and tokens required to reach a fixed threshold
Oracle stored forgetting	Increase from each world's best historical oracle-address NLL
Routed forgetting	Same measurement using task-free prefix routing
Forward transfer	Tokens saved on B, C, D relative to a fresh base-rooted adapter

Property	Measurement
Compositional reuse	Zero-shot and post-adaptation gain of $A - B + C$ on D
Negative transfer	Effect of prior related worlds on unrelated E
Return recall	Performance on A immediately after E, before any refresh
Memory	Frozen edge parameters and metadata bytes
Addressing cost	Prefix tokens × number of candidate addresses evaluated
Sequential/joint gap	Sequential VAMP or mutable LoRA versus shuffled joint adapter

The most interesting single plot will probably be the D adaptation curve:

1. frozen base
2. best single prior edge
3. $A - B + C$ composition
4. composition plus trained residual
5. independently trained D adapter
6. joint-data adapter

If the composition is useful before training or reaches the target loss with fewer D tokens, you have direct evidence of reusable LoRA-edge structure.

---

8. LoRA memory scale

Assuming your planned eight-layer, $d_{\text{model}} = 256$ model and LoRA on W_Q and W_V only:

For one 256×256 projection, rank r uses:

$$r(256 + 256) = 512r$$

parameters.

Across two projections and eight layers:

$$8 \times 2 \times 512r = 8192r.$$

Therefore:

Rank	Parameters per edge	BF16 stored weights
$r = 2$	16,384	32 KiB
$r = 4$	32,768	64 KiB

The VAMP graph containing A, B, C, E, ρ_D would therefore use approximately:

$$5 \times 64\ \mathrm{KiB} = 320\ \mathrm{KiB}$$

of raw rank-4 adapter weights, excluding small metadata.

A BF16 checkpoint of an 8–12M-parameter base is approximately 16–24 MB, so the entire five-edge graph is still roughly 50–75 times smaller than storing five full parameter states. An individual edge is roughly 250–375 times smaller than one full checkpoint.

If you later add W_K and W_O, double those adapter figures.

---

9. Calibration gate before committing to the final split

The largest uncertainty is not compute; it is whether the 8M model already generalizes too well across the missing recipe cells.

Run a two-epoch reduced-base pilot and measure the held-out conjunction gap against the matched controls.

Reasonable initial go/no-go heuristics are:

- Mean held-out gap: roughly 0.08–0.30 nats/token.
- Each selected world: at least 0.05 nats/token above its matched control.
- Held-in validation loss: within approximately 0.05 nats/token of the comparable full-corpus base.
- Mutable sequential LoRA: measurable degradation on at least two old worlds after the full stream.

These are engineering thresholds, not established benchmark constants.

If the gap is too small, change only one knob:

$$8 \times 8 \longrightarrow 6 \times 6.$$

That increases each missing conjunction from $1/64$ to $1/36$ of the corpus and removes a larger cross-product of noun–verb co-occurrences.

If the gap is excessively large, use 10×10 or 12×12.

Do not introduce adjective conjunctions, semantic clusters, generated supplements, or feature-conditioned worlds until this simplest knob has been tried.

Time and cost estimates

Partitioning

The official data involved are only a few gigabytes—the principal train file is approximately 1.9 GB, the GPT-4 train variant approximately 2.2 GB, and the metadata tar approximately 1.61 GB. 🤗 Hugging Face +1

On a normal NVMe workstation:

Operation	Machine time
Download metadata, when not cached	2–20 min
Unpack and stream metadata	5–20 min
Hash-join to canonical corpus	5–20 min
Count and balance word buckets	Under 5 min
Group duplicates and assign splits	5–15 min
Tokenize and write new shards	10–45 min
Produce counts and audit tables	Under 10 min
Typical total	**30–90 min**

Operation	Machine time
Conservative first run	**Up to about 2 hours**

There is no API or inference cost.

The initial implementation is probably **4–8 hours of engineering**, assuming your existing loader and tokenizer can be reused. Auditing and fixing metadata-match failures may add another 1–3 hours. Subsequent repartitions from cached metadata should take approximately 5–20 minutes.

Expected electricity cost is only a few cents. Working storage should remain comfortably under roughly 10–20 GB including unpacked metadata, tokenized base/world shards, and temporary files.

Reduced-base retraining on the RTX 4090

Five held-out cells leave approximately 429–437M training tokens per epoch.

Using the previously estimated throughput for your JAX TinyStories-8M setup of approximately 150k–400k tokens/s:

$$T_{\text{epoch}} = \frac{429\text{M}–437\text{M}}{150\text{k}–400\text{k tokens/s}} \approx 18–49 \text{ minutes}.$$

Allowing for compilation, validation, checkpointing, and data-loader stalls:

Training amount	Raw compute	Realistic wall time
1 epoch	18–49 min	25–65 min
2-epoch calibration	36–97 min	0.8–2.2 h
5 epochs	1.5–4.1 h	**2–5 h**
10 epochs	3.0–8.1 h	**4–10 h**
20 epochs	6.0–16.2 h	**7–19 h**

I would use **five epochs with validation-based checkpoint selection** for the first real base. Extend to ten only if the held-in validation curve is clearly

still improving.

That is consistent with the TinyStories paper's general result that sub-10M models can be trained on a single GPU in less than a day. arXiv

Assuming:

- total workstation draw: 450–650 W
- electricity: $0.15/kWh

the approximate electricity cost is:

Base run	Approximate electricity cost
2-epoch calibration	$0.05–$0.22
5 epochs	**$0.14–$0.49**
10 epochs	$0.27–$0.98
20 epochs	$0.47–$1.85

These estimates assume your paper-faithful approximately 8–12M parameter, 10k-vocabulary model. Accidentally using the hosted GPT-Neo-style 50,257-token vocabulary would increase embedding and output-softmax costs, so benchmark the actual token throughput before scheduling the full run.

Continual LoRA experiment time

One 80% world-training split contains approximately 5.8–5.9M tokens.

At the same raw throughput, one pass is only about 15–40 seconds of pure token compute, but compilation, evaluation, optimizer initialization, and checkpointing will dominate. A realistic expectation is:

- 3–10 minutes per trained edge
- 30–90 minutes for one rank-4 sequential experiment with all evaluations
- 1–3 hours for the mutable, independent-bank, and VAMP methods at one seed
- approximately 6–12 GPU hours for ranks 2 and 4, three seeds, and the joint reference run

At the same power assumptions, the complete multi-seed LoRA campaign should cost roughly $0.40–$1.20 in electricity.

Concrete first milestone

The first milestone should contain exactly these artifacts:

1. A frozen partition manifest containing corpus hash, normalization version, bucket word lists, story hashes, and split assignments.
2. Reduced-base train and validation shards.
3. Five world train/validation/test shards.
4. Matched-control shards for each world.
5. A partition audit showing token counts, feature distributions, source distributions, length distributions, exact-duplicate checks, and prefix recipe-word visibility.
6. One five-epoch reduced base.
7. One rank-4 run each for mutable LoRA, independent frozen adapters, and VAMP composition/routing.
8. A loss matrix after every episode and the D composition/adaptation plot.

This should be approximately **one to two focused engineering days**, followed by **several hours of unattended GPU time**, with essentially zero data-generation expense.

The resulting claim would be appropriately narrow but meaningful:

> On genuine TinyStories text, a VAMP graph of frozen low-rank edges retains prior subdistribution competence, retrieves it task-free from observed prefixes, and reuses prior edges to accelerate acquisition of a held-out factorial combination, while explicitly accounting for memory and addressing cost.

That is enough to validate the basic VAMP/LoRA mechanism before returning to anything resembling the original full TinyWorlds proposal.